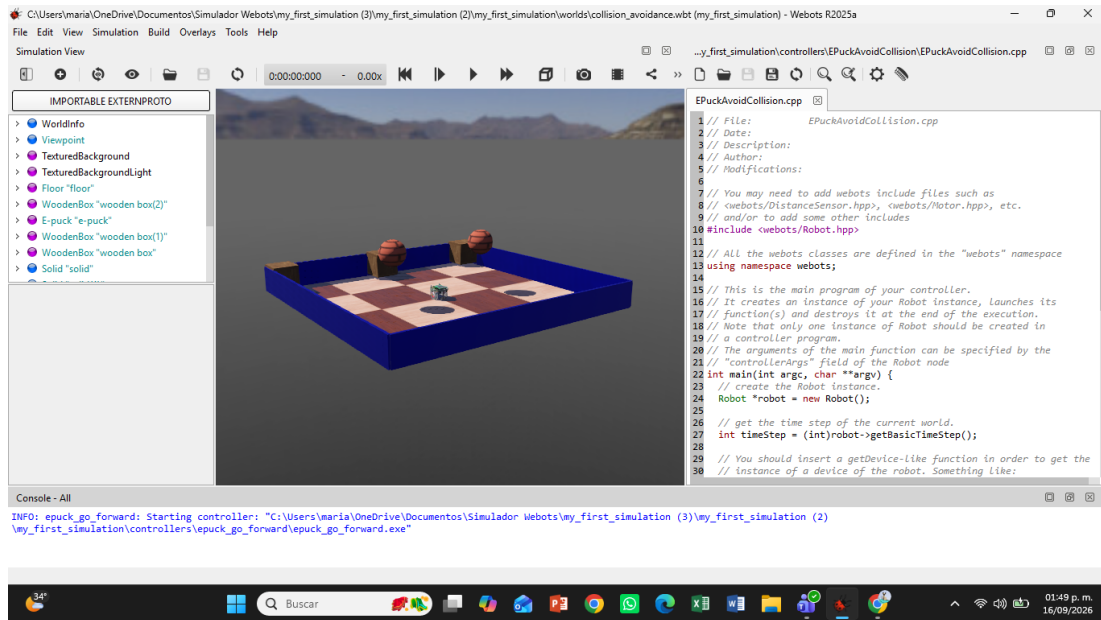


Tutorial 4

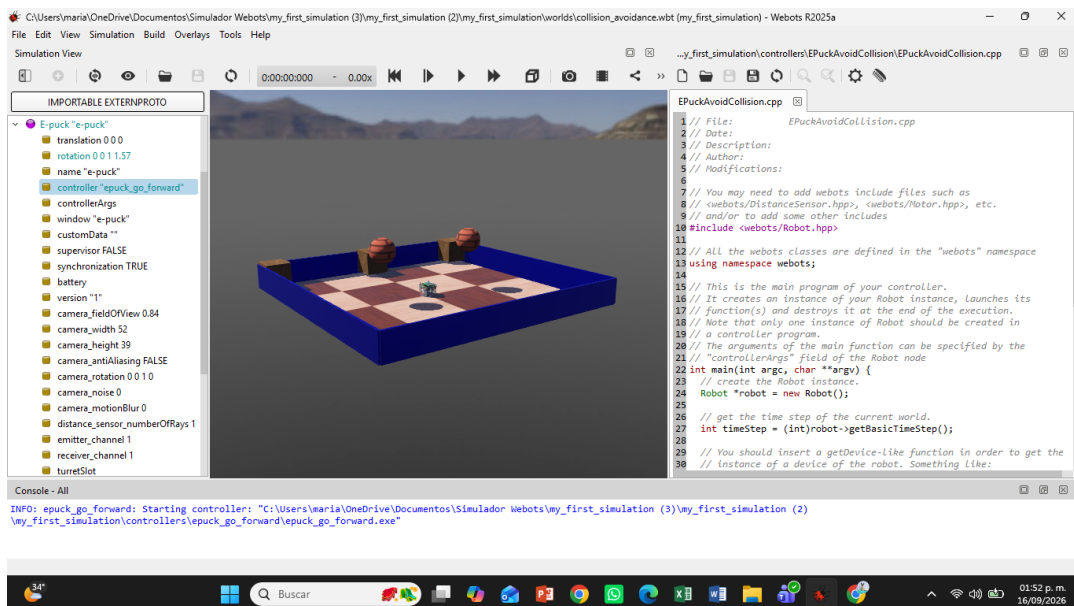
Primero lo que se hizo fue guarda el mundo anterior como collision_avoidance.wbt.

Después lo que se hizo fue cambiar el lenguaje a C++.

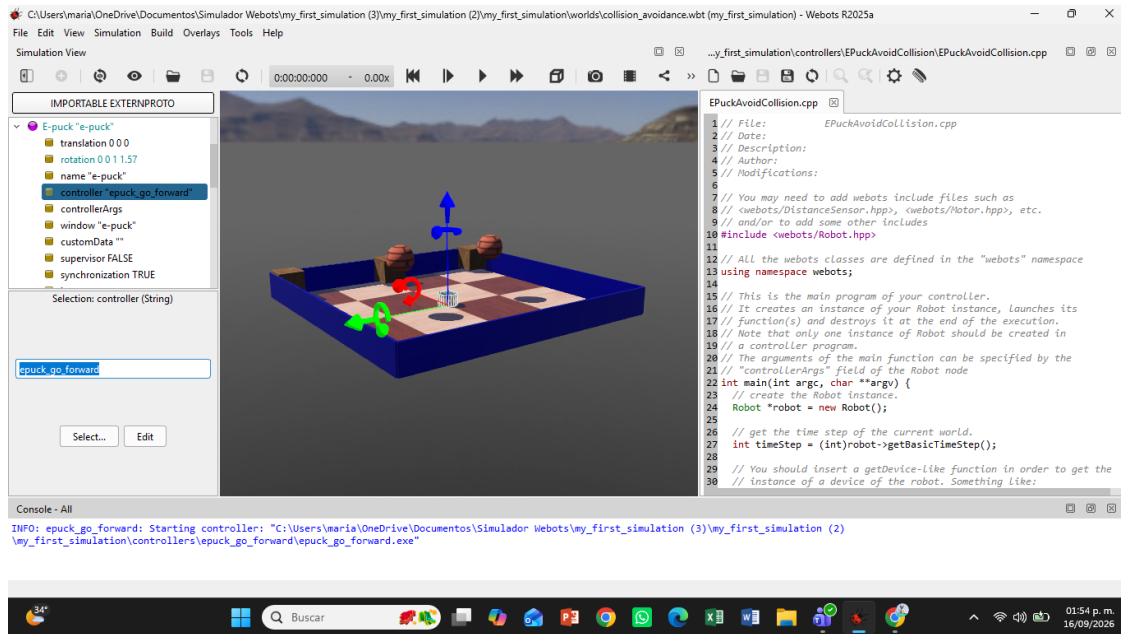
(No hay captura porque se me olvido tomarla)



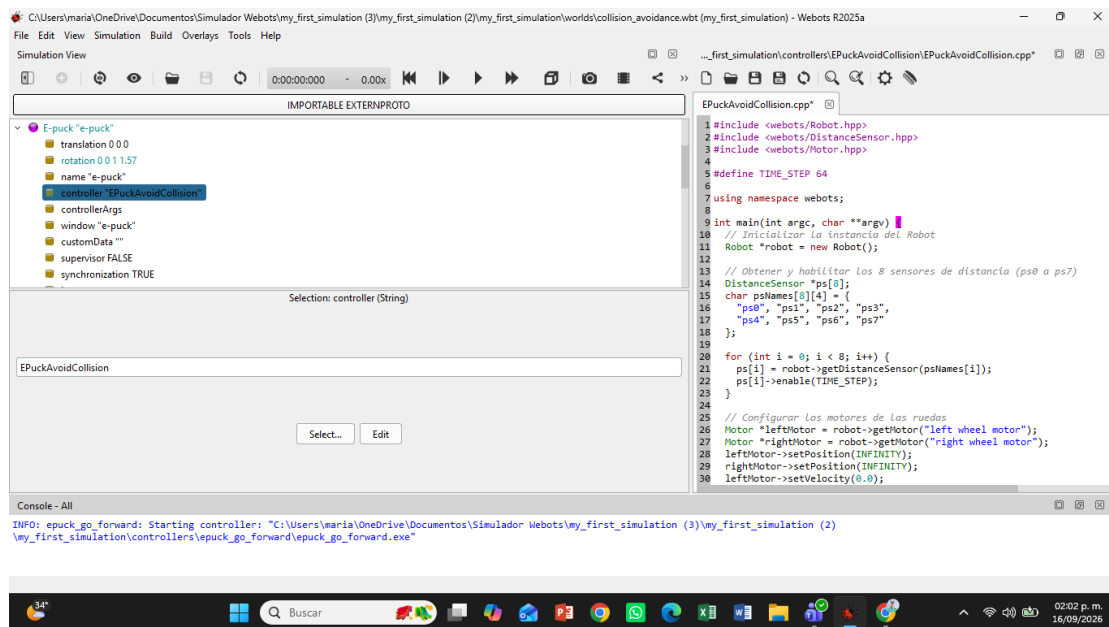
Se abrió la escena de trabajo collision_avoidance.wbt en Webots, en la cual se observa el robot E-puck ubicado al centro del escenario de simulación. Al mismo tiempo, en la ventana de la derecha, se creó el nuevo archivo de código EPuckAvoidCollision.cpp con la plantilla base en C++ para comenzar a programar la lógica del robot.



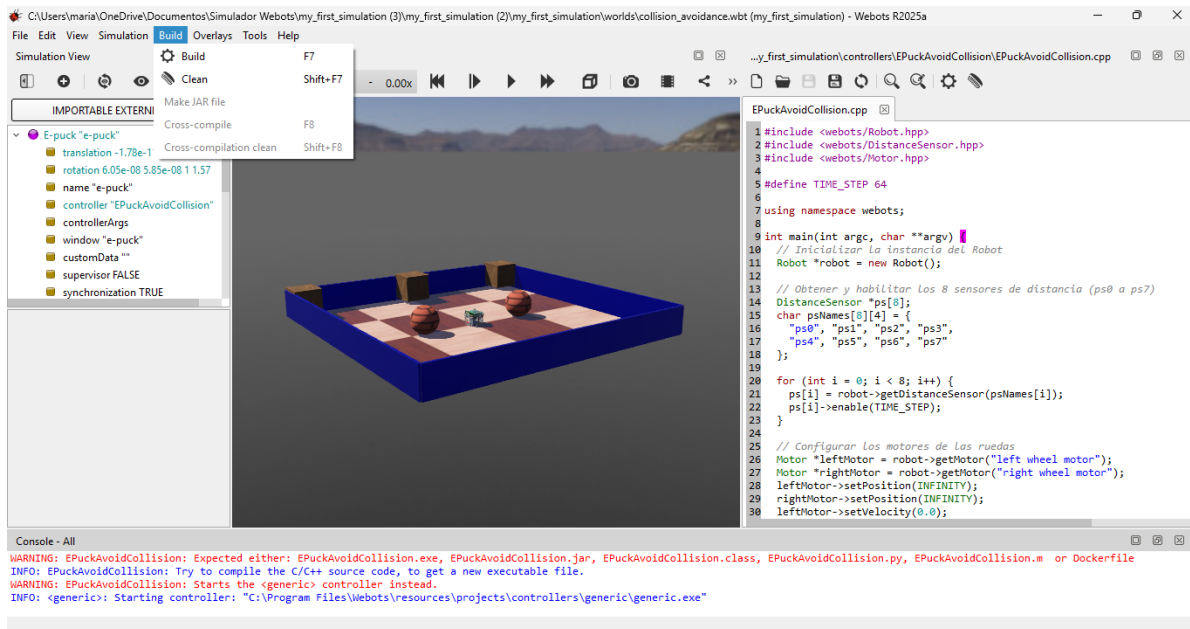
Se desplegó el árbol de la escena (Scene Tree) a la izquierda, expandiendo el nodo del robot "e-puck". Se seleccionó la propiedad controller, la cual en ese momento aún tenía asignado el controlador anterior epuck_go_forward. Este paso sirve para ubicar la casilla donde se asignará el nuevo controlador personalizado.



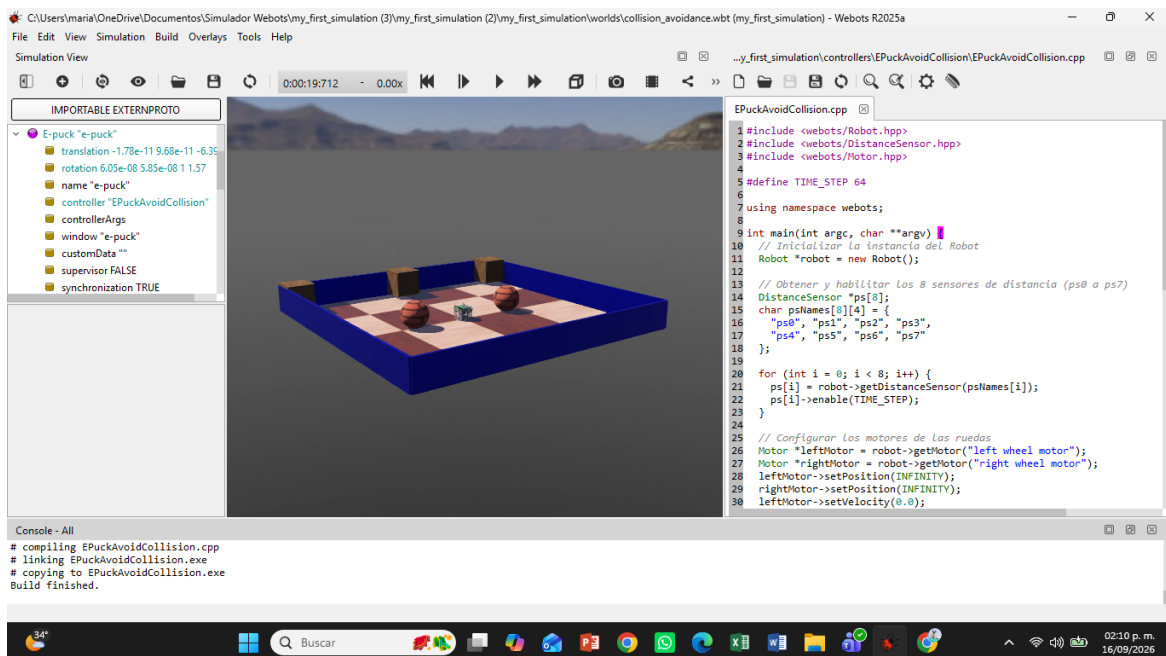
Se abrió el campo de texto de la propiedad controller en el panel inferior del árbol de la escena (Selection: controller). En este cuadro de texto se seleccionó el nombre del controlador actual (epuck_go_forward) para sobrescribirlo y reemplazarlo por el nuevo controlador EPuckAvoidCollision.



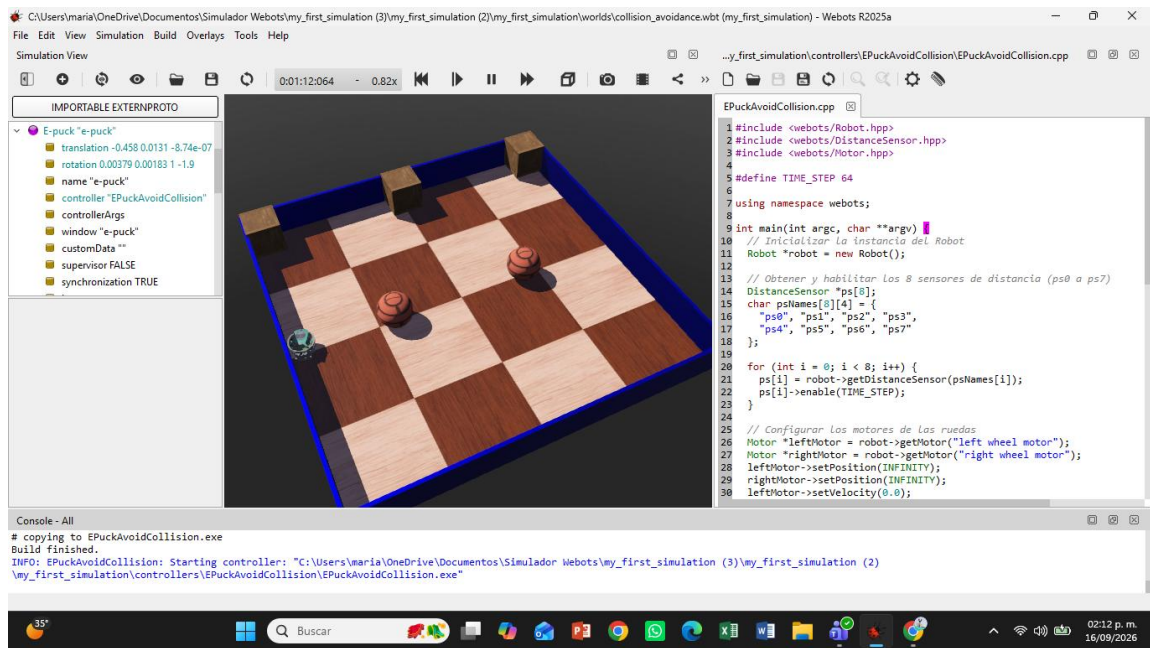
Se confirmó el cambio de controlador asignando exitosamente EPuckAvoidCollision al robot E-puck en el árbol de la escena. Asimismo, en la ventana del editor de código a la derecha, se redactó la programación en C++ para incluir las librerías del robot, sensores de distancia y motores, además de inicializar y habilitar los 8 sensores de distancia infrarrojos (ps0 a ps7) junto con los motores de las ruedas.



Se accedió al menú superior Build y se seleccionó la opción Build para iniciar el proceso de compilación del controlador en C++ (EPuckAvoidCollision.cpp). Este procedimiento se realizó para transformar el código fuente en un archivo ejecutable que el entorno de Webots pueda procesar.



Se finalizó el proceso de compilación del controlador C++ (EPuckAvoidCollision.cpp). En la consola inferior se verificó la correcta ejecución de los comandos de compilación y enlazado, concluyendo con el mensaje Build finished. Esto confirmó que el ejecutable EPuckAvoidCollision.exe se generó correctamente y sin errores de sintaxis, quedando listo para ser ejecutado en la simulación.



Se inició la simulación en Webots presionando el botón Play. En la consola se confirmó la puesta en marcha del programa con el mensaje INFO: EPuckAvoidCollision: Starting controller.... En el visor 3D se observó el funcionamiento autónomo del robot E-puck, el cual avanzó desde su posición inicial hasta la esquina de la arena y realizó un giro hacia la derecha al detectar la pared azul con sus sensores de distancia, evitando exitosamente la colisión.